Chapter 5.4: Measuring and Improving Cache Performance

Associativity, Multilevel Caches, and Blocking

Contents

1 Cache Performance Equations

1.1 CPU Time Breakdown

CPU time can be divided into two components:

- **CPU execution cycles:** Time spent executing the program
- **Memory-stall cycles:** Time spent waiting for the memory system

CPU Time Formula

$$\text{CPU Time} = (\text{CPU execution cycles} + \text{Memory-stall cycles}) \times \text{Clock cycle time}$$

1.2 Memory-Stall Cycles

Memory stalls come primarily from **cache misses**:

Memory-Stall Cycles

$$\text{Memory-stall cycles} = \text{Read-stall cycles} + \text{Write-stall cycles}$$

For reads:

$$\text{Read-stall cycles} = \frac{\text{Reads}}{\text{Program}} \times \text{Read miss rate} \times \text{Read miss penalty}$$

For writes (write-through):

$$\text{Write-stall cycles} = \frac{\text{Writes}}{\text{Program}} \times \text{Write miss rate} \times \text{Write miss penalty} + \text{Write buffer stalls}$$

1.3 Simplified Formula

If read/write penalties are equal and write buffer stalls are negligible:

Simplified Memory-Stall Formula

$$\text{Memory-stall cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

Or per instruction:

$$\text{Memory-stall cycles/Instruction} = \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

1.4 Average Memory Access Time (AMAT)

Average Memory Access Time

$$\text{AMAT} = \text{Hit time} + (\text{Miss rate} \times \text{Miss penalty})$$

This metric captures performance for **both hits and misses**.

Example: Calculating AMAT

Given:

- Clock cycle = 1 ns
- Miss penalty = 20 cycles
- Miss rate = 0.05 misses/instruction
- Hit time = 1 cycle

Solution:

$$\text{AMAT} = 1 + (0.05 \times 20) = 1 + 1 = 2 \text{ cycles} = 2 \text{ ns}$$

Amdahl's Law Reminder

If the processor is made faster but the memory system is not improved, the **fraction of time spent on memory stalls will increase**.
Example: Reducing CPI from 2 to 1 with 3.44 memory stall cycles:

- Before: Memory stalls = $\frac{3.44}{2+3.44} = 63\%$ of execution time
- After: Memory stalls = $\frac{3.44}{1+3.44} = 78\%$ of execution time

2 Reducing Miss Rate: Associativity

There are three block placement schemes:

2.1 1. Direct-Mapped Cache

Direct-Mapped (1-way Set Associative)

Each memory block maps to **exactly one** cache location.

$$\text{Cache block} = (\text{Block address}) \mod (\text{Number of blocks})$$

2.2 2. Fully Associative Cache

Fully Associative (m-way for m blocks)

A block can be placed in **any location** in the cache.

- All entries must be searched in parallel
- Requires a comparator per cache entry
- Practical only for **small caches** (high hardware cost)

2.3 3. Set-Associative Cache

Set-Associative (n-way)

A block can be placed in a **fixed number of locations** (n locations per set).

$$\text{Set} = (\text{Block address}) \mod (\text{Number of sets})$$

Process:

1. Block is **directly mapped to a set** (using index)
2. All blocks **within the set are searched** in parallel

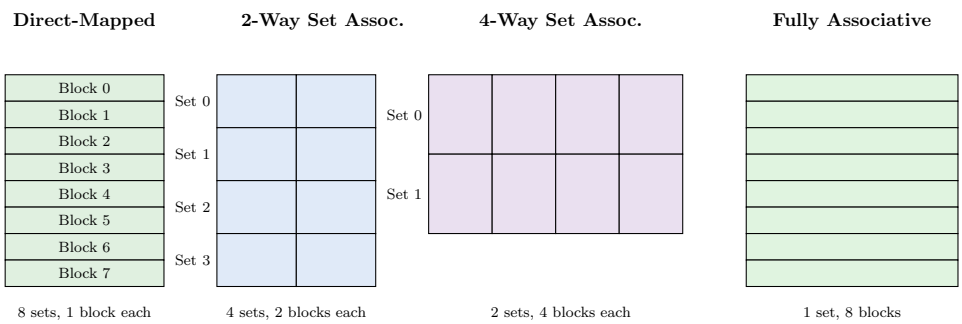

Figure 1: **Figure 5.15:** An 8-block cache configured with different associativities.

2.4 Trade-offs of Associativity

- **Advantage:** Higher associativity → **lower miss rate**
- **Disadvantage:** Higher associativity → **higher hit time** (more comparators, multiplexors)

Effect on address fields (for fixed cache size):

- Doubling associativity → halves number of sets
- Index field decreases by 1 bit
- Tag field increases by 1 bit

2.5 Hardware for Set-Associative Caches

A 4-way set-associative cache requires:

- **4 parallel comparators** (one per way)
- **4-to-1 multiplexor** to select the matching block

Content Addressable Memory (CAM)

A circuit that combines **comparison and storage**. Instead of providing an address to read data, you send data and the CAM returns the matching index.
Usage: 8-way and higher associativity often use CAMs.

3 Block Replacement Policies

When a miss occurs in an associative cache, which block should be replaced?

Least Recently Used (LRU)

The block replaced is the one that has been **unused for the longest time**.
Implementation:

- 2-way: Single bit per set (set on each access)
- Higher associativity: More complex tracking required

Example: Associativity Comparison

Sequence: 0, 8, 0, 6, 8 (block addresses)
Direct-mapped (4 blocks):

- Blocks 0, 8 map to block 0; Block 6 maps to block 2
- Result: **5 misses** (0: miss, 8: miss/replace 0, 0: miss/replace 8, 6: miss, 8: miss/replace 0)

2-way set associative (2 sets $\times$ 2 blocks):

- All map to set 0
- Using LRU replacement
- Result: **4 misses**

Fully associative (4 blocks):

- Result: **3 misses** (minimum possible for 3 unique addresses)

4 Reducing Miss Penalty: Multilevel Caches

Multilevel Cache

A memory hierarchy with **multiple levels of caches** (L1, L2, L3...) rather than just a single cache and main memory.

4.1 How It Works

1. Access L1 (primary) cache first
2. On L1 miss, access L2 (secondary) cache
3. On L2 miss, access main memory

Key benefit: If data is in L2, the miss penalty is the L2 access time (much shorter than main memory).

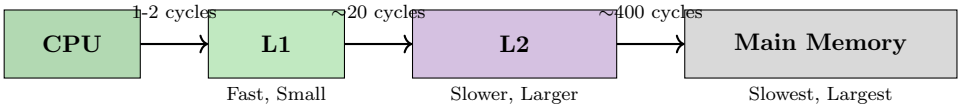

Figure 2: Multilevel cache hierarchy with typical access times.

4.2 Performance Formula

Total CPI with Multilevel Cache

$$\text{Total CPI} = \text{Base CPI} + \text{L1 stalls/instruction} + \text{L2 stalls/instruction}$$

Where:

- L1 stalls = L1 miss rate × L2 access time
- L2 stalls = Global miss rate × Memory access time

Example: Multilevel Cache Performance

Given:

- Base CPI = 1.0, Clock rate = 4 GHz
- Memory access time = 100 ns = 400 cycles
- L1 miss rate = 2%
- L2 access time = 5 ns = 20 cycles
- Global miss rate (to memory) = 0.5%

Single-level cache:

$$\text{CPI} = 1.0 + 0.02 \times 400 = 1.0 + 8 = 9.0$$

Two-level cache:

$$\begin{aligned} \text{L1 stalls} &= 0.02 \times 20 = 0.4 \\ \text{L2 stalls} &= 0.005 \times (20 + 400) = 2.1 \\ \text{Total CPI} &= 1.0 + 0.4 + 2.1 = 3.5 \end{aligned}$$

Speedup: $\frac{9.0}{3.5} = 2.6\times$ faster!

4.3 Design Considerations

Property	L1 Cache	L2 Cache
Primary Goal	Minimize **hit time**	Minimize **miss rate**
Size	Smaller	Much larger
Block Size	Smaller	Larger
Associativity	Lower	Higher

Table 1: Design focus for L1 vs L2 caches.

4.4 Global vs. Local Miss Rate

Miss Rate Terminology

Global Miss Rate: Fraction of references that miss in **all cache levels**.
Local Miss Rate: Fraction of references to one cache level that miss **at that level**.
Note: Local miss rate of L2 is much higher than global miss rate because L1 filters out easy-to-hit references.

5 Software Optimization: Blocking

Blocking (Tiling)

Instead of operating on entire rows or columns of arrays, blocked algorithms operate on **submatrices** to maximize data reuse within the cache.
Goal: Improve **temporal locality** by accessing data multiple times before it is evicted.

5.1 Why Blocking Helps

For matrix multiplication without blocking:

- Reads all $N \times N$ elements of B
- Reads same row of A repeatedly
- May cause many cache misses if matrices don't fit in cache

With blocking (BLOCKSIZE $\times$ BLOCKSIZE submatrices):

- Works on small submatrices that fit in cache
- Accesses reduced from $2N^3 + N^2$ to approximately $\frac{2N^3}{\text{BLOCKSIZE}} + N^2$
- Improvement factor $\approx$ BLOCKSIZE

5.2 Blocking Exploits Both Localities

- **Spatial locality:** Elements of A accessed sequentially
- **Temporal locality:** Elements of B reused within the block

Algorithm vs. Cache Performance

Algorithms with fewer operations may perform **worse** if they have poor cache behavior!
Example: Radix Sort executes fewer instructions than Quicksort for large arrays, but Quicksort has **far fewer cache misses** and often runs faster in practice.

Autotuning

An approach where numerical libraries **parameterize their algorithms** and search at runtime to find the best combination (e.g., BLOCKSIZE) for a particular computer's memory hierarchy.

6 Check Yourself

Which is generally true about multilevel cache design?

1. First-level caches are more concerned about hit time, and second-level caches are more concerned about miss rate.
   TRUE – L1 focuses on speed; L2 focuses on catching misses.
2. First-level caches are more concerned about miss rate, and second-level caches are more concerned about hit time.
   FALSE

7 Flashcards

1. **Q: What are the two primary techniques for improving cache performance?**
   A: Reducing the miss rate and reducing the miss penalty.

2. **Q: What is the formula for CPU time in terms of execution cycles and memory-stall cycles?**
   A: CPU time = (CPU execution cycles + Memory-stall cycles) × Clock cycle time.

3. **Q: Memory-stall clock cycles are primarily caused by what events?**
   A: Cache misses.

4. **Q: What is the formula for Average Memory Access Time (AMAT)?**
   A: AMAT = Hit time + (Miss rate × Miss penalty).

5. **Q: According to Amdahl's Law, what happens if the processor gets faster but memory doesn't?**
   A: The fraction of time spent on memory stalls will increase.

6. **Q: What is a fully associative cache?**
   A: A cache structure in which a block can be placed in any location in the cache.

7. **Q: What is a set-associative cache?**
   A: A cache with a fixed number of locations (at least two) where each block can be placed.

8. **Q: How is the set for a memory block determined in a set-associative cache?**
   A: Set = (Block number) modulo (Number of sets in the cache).

9. **Q: A direct-mapped cache is equivalent to what type of set-associative cache?**
   A: A 1-way set-associative cache.

10. **Q: A fully associative cache with m entries is equivalent to what?**
    A: An m-way set-associative cache with only one set.

11. **Q: What is the primary advantage of increasing associativity?**
    A: It usually decreases the miss rate.

12. **Q: What is the main disadvantage of increasing associativity?**
    A: A potential increase in hit time due to more complex hardware.

13. **Q: How does doubling associativity affect the index and tag fields (fixed cache size)?**
    A: Index decreases by 1 bit; tag increases by 1 bit.

14. **Q: What hardware is needed for a 4-way set-associative cache vs. direct-mapped?**
    A: Four parallel comparators and a 4-to-1 multiplexor.

15. **Q: What is a Content Addressable Memory (CAM)?**
    A: A circuit that combines comparison and storage, returning the index of a matching entry.

16. **Q: What is the most commonly used block replacement scheme?**
    A: Least Recently Used (LRU).

17. **Q: Define Least Recently Used (LRU).**
    A: The block replaced is the one unused for the longest time.

18. **Q: What is a multilevel cache?**
    A: A memory hierarchy with multiple levels of caches rather than just one cache and main memory.

19. **Q: How does a second-level cache reduce miss penalty?**
    A: If data is in L2, the penalty is only the L2 access time, much less than main memory.

20. **Q: What is the design focus of L1 caches in a multilevel system?**
    A: Minimizing hit time to enable shorter clock cycles.

21. **Q: What is the design focus of L2 caches in a multilevel system?**
    A: Minimizing miss rate to reduce the penalty of main memory access.

22. **Q: Define global miss rate.**
    A: The fraction of references that miss in all levels of a multilevel cache.

23. **Q: Define local miss rate.**
A: The fraction of references to one cache level that miss at that level.

24. **Q: Why is the local miss rate of L2 higher than the global miss rate?**
A: Because L1 filters out most accesses with good locality, leaving harder-to-hit references.

25. **Q: What is the goal of blocking (tiling) in software optimization?**
A: To maximize data reuse in the cache before data is replaced, improving temporal locality.

26. **Q: What does blocking operate on instead of entire rows or columns?**
A: Submatrices or blocks.

27. **Q: What two types of locality does blocking exploit?**
A: Spatial locality and temporal locality.

28. **Q: By what factor does blocking reduce memory accesses (approximately)?**
A: By a factor of BLOCKSIZE.

29. **Q: Why might Quicksort outperform Radix Sort in practice for large arrays?**
A: Quicksort has significantly fewer cache misses.

30. **Q: What is autotuning?**
A: An approach where libraries parameterize algorithms and search at runtime for the best parameters.

31. **Q: What is the formula for memory-stall cycles per instruction?**
A: (Misses/Instruction) $\times$ Miss penalty.

32. **Q: For write-through caches, what are the two sources of write stalls?**
A: Write misses and write buffer stalls.

33. **Q: When can write buffer stalls be safely ignored?**
A: When the buffer is deep enough and memory can accept writes faster than average write frequency.

34. **Q: What is a potential negative consequence of increasing cache size?**
A: It can increase hit time, potentially increasing cycle time or pipeline stages.

35. **Q: In a 4-way set-associative cache with 4096 blocks, how many sets are there?**
A: 4096 / 4 = 1024 sets.

36. **Q: How many index bits are used in a fully associative cache?**
A: Zero – there is no index; the entire cache is searched.

37. **Q: What three portions make up an address for a set-associative cache?**
A: Tag, index, and block offset.

38. **Q: What is the formula for Total CPI with two-level caching?**
A: Base CPI + L1 stalls/instruction + L2 stalls/instruction.

39. **Q: As associativity increases, what happens to LRU implementation complexity?**
A: It becomes harder to implement.

40. **Q: Which cache designer guideline about memory bandwidth is generally valid?**
A: The higher the memory bandwidth, the larger the cache block can be.